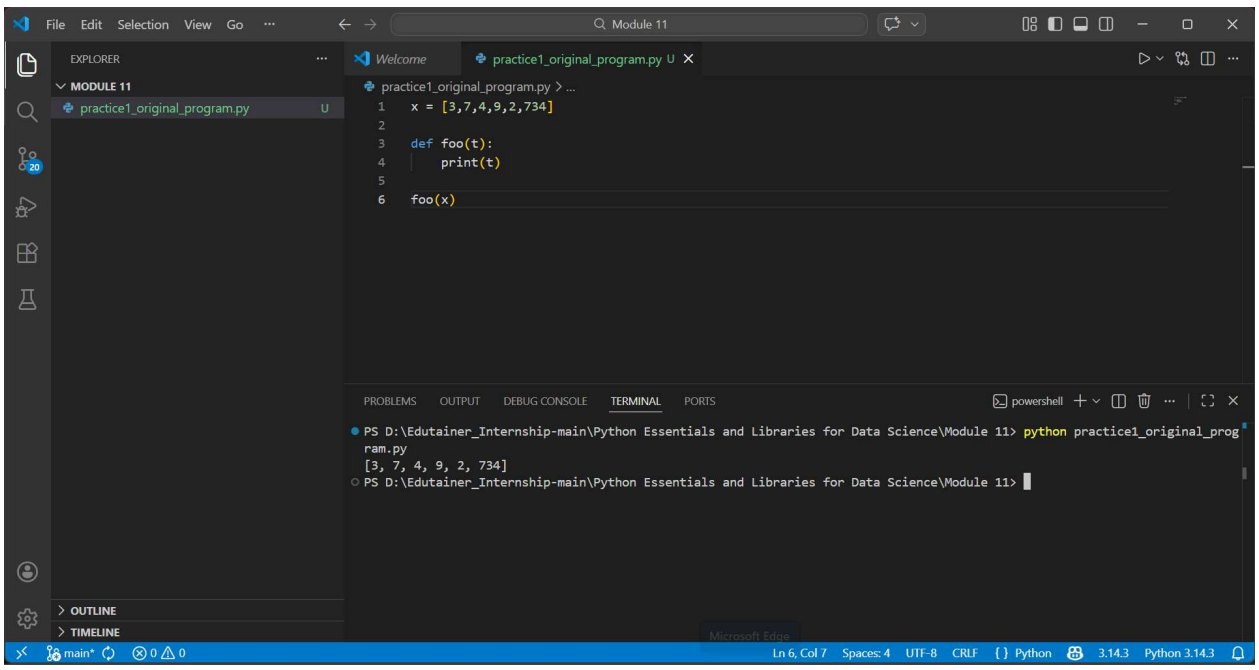
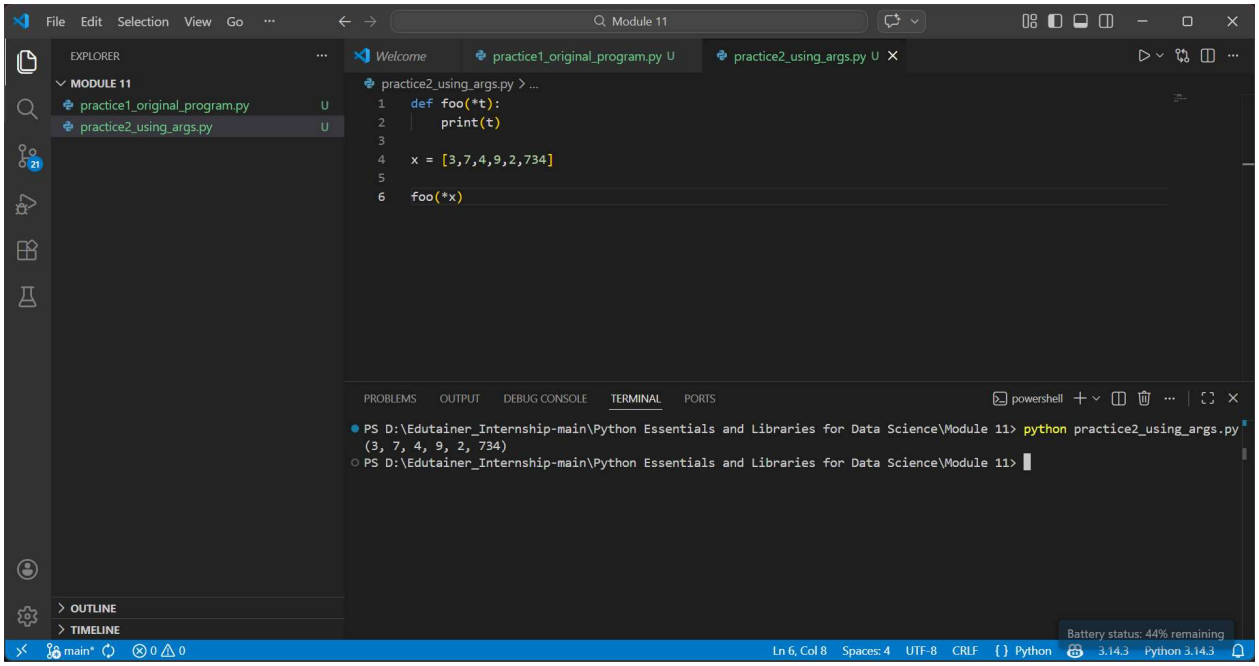
Parameter Types in Python Functions

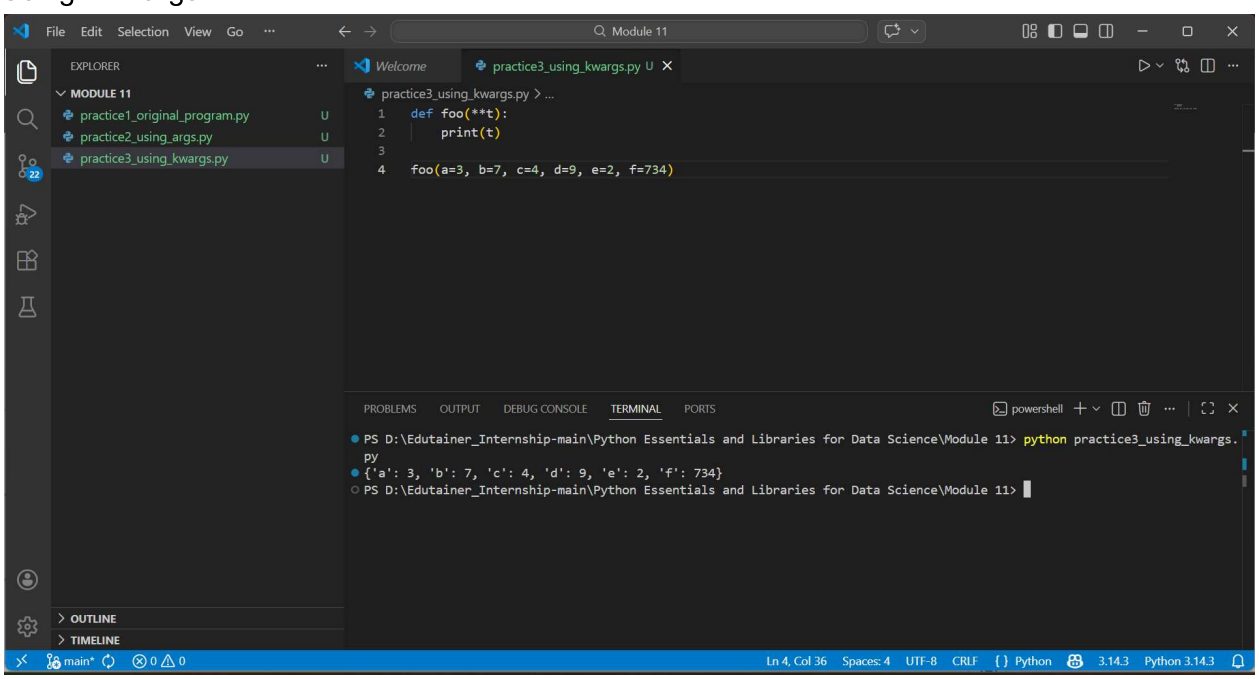
- When passing parameters to functions, there are special types of arguments provided by Python:
    - `*args` (arguments)
    - `**kwargs` (keyword arguments)
- `*args` is used to pass an **unlimited number of parameters** separated by commas.
- `**kwargs` is used to pass an **unlimited number of parameters with keywords** for each value.



Using *args:-



Using **kwargs:-



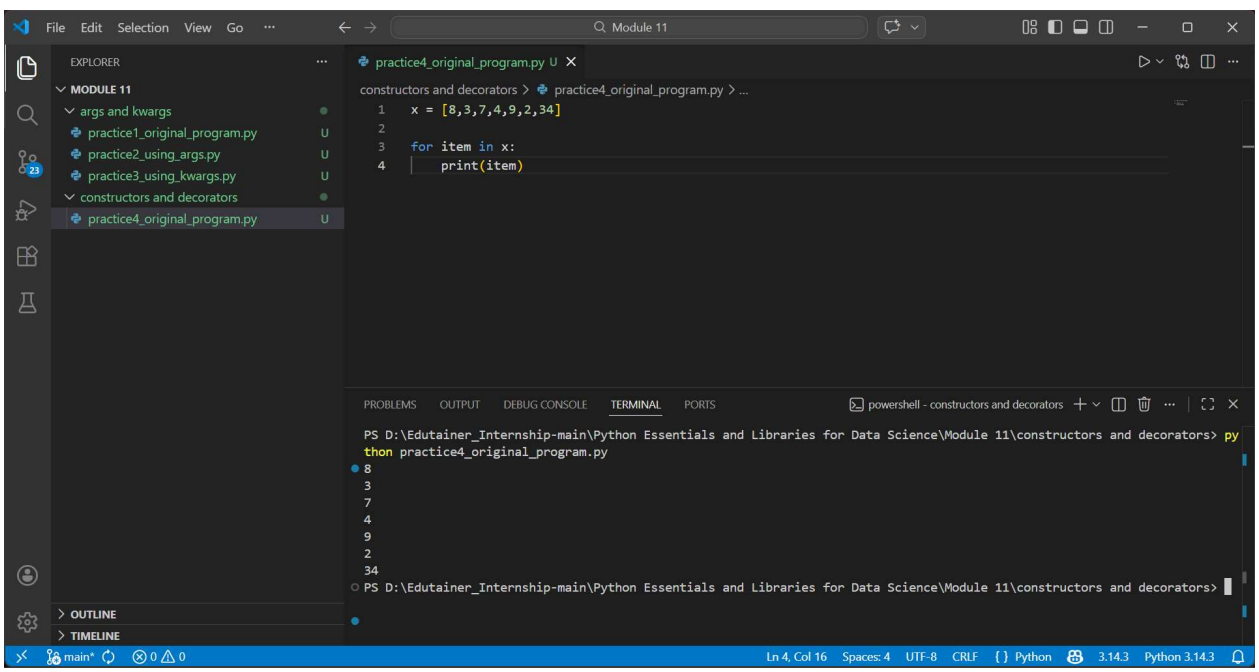
Python Generators and Decorators:-

Generators in Python

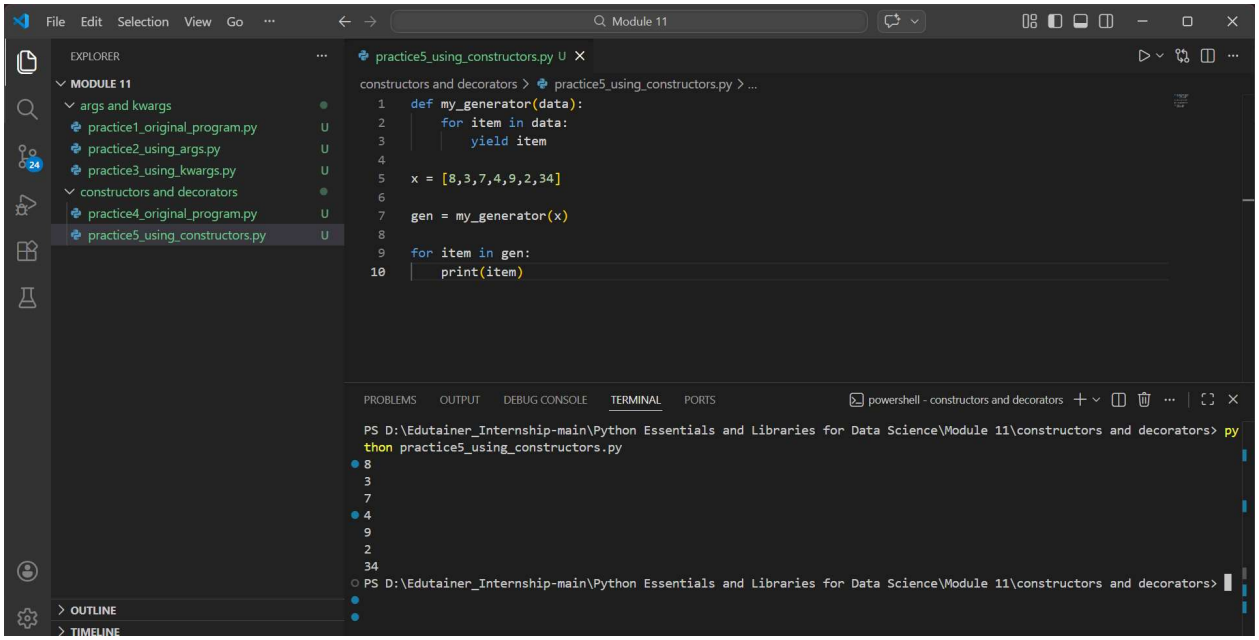
- A **generator** in Python provides a way to create **iterators.**
- They give us **total control on iterating through values.**
- Using generators, we can work on data items **one at a time.**
    - Generators use a special keyword called `yield` to produce output.
    - Also uses `next()` function to iterate to the next output or item.

Decorators in Python

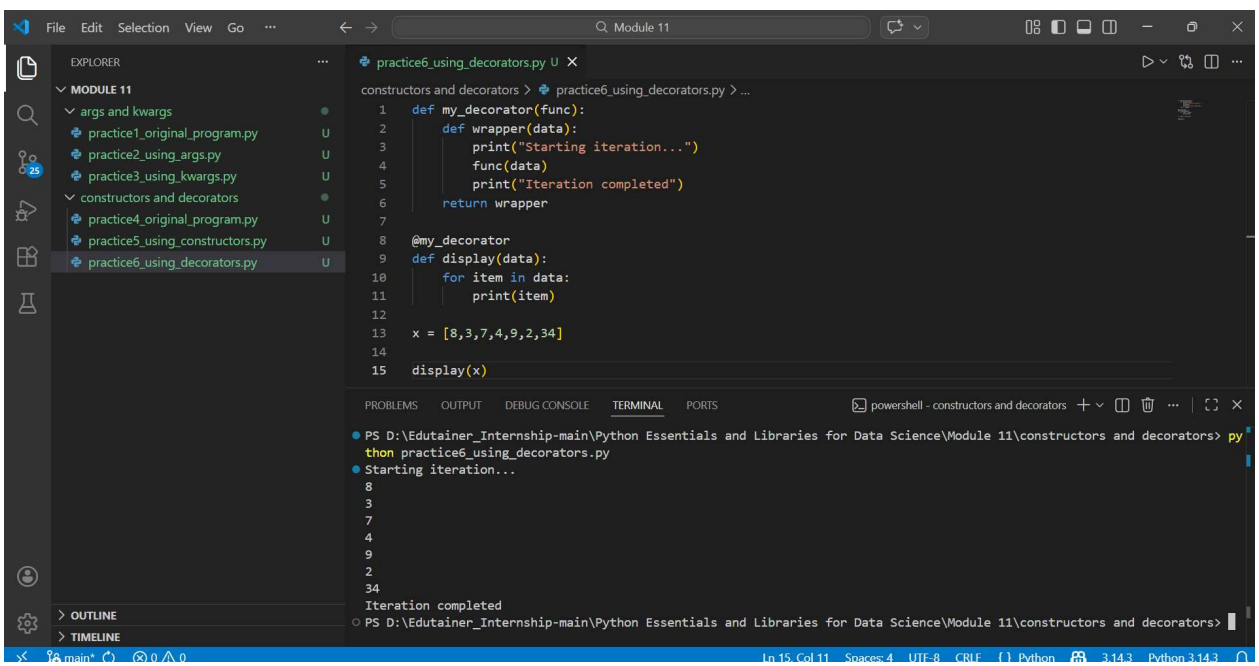
- A **decorator** is a method of giving **extra functionality to an existing function** without modifying the original function.
- With decorators, we can handle **user inputs in a much better way.**



Using constructors:-



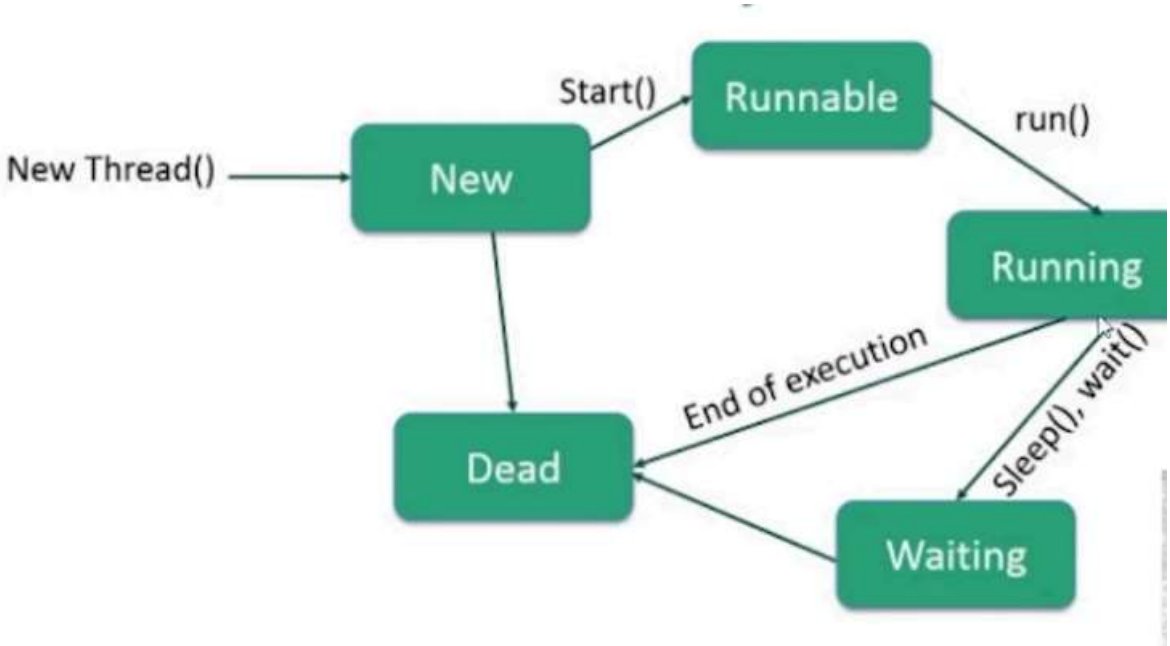
Using decorators:-

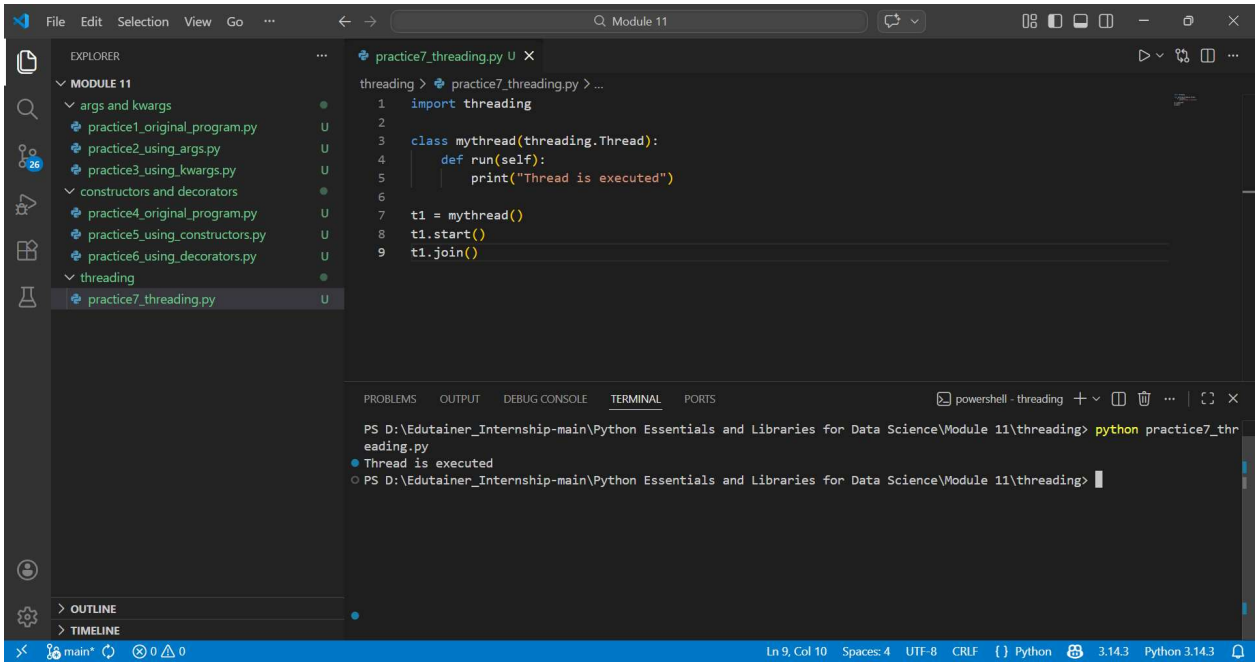


Python Multithreading:-

Thread in Python

- A **thread** is a class provided by the `threading` module in Python that enables **multitasking.**
- A thread is the **smallest unit of execution.**
    - It cannot be divided further.
    - It represents a single flow of execution in a program.
- A **process** is a collection of threads.
    - A process can have:
        - **Single thread** (single-task execution)
        - **Multiple threads** (multi-task execution)





Thread Class Functions

- **run()**
  → Contains the code that the thread executes

- **start()**
  → Starts the thread

- **setName()**
  → Used to assign a name to the thread object

- **getName()**
  → Used to retrieve the thread name

- **join()**
  → Ensures the thread completes its work before another thread continues

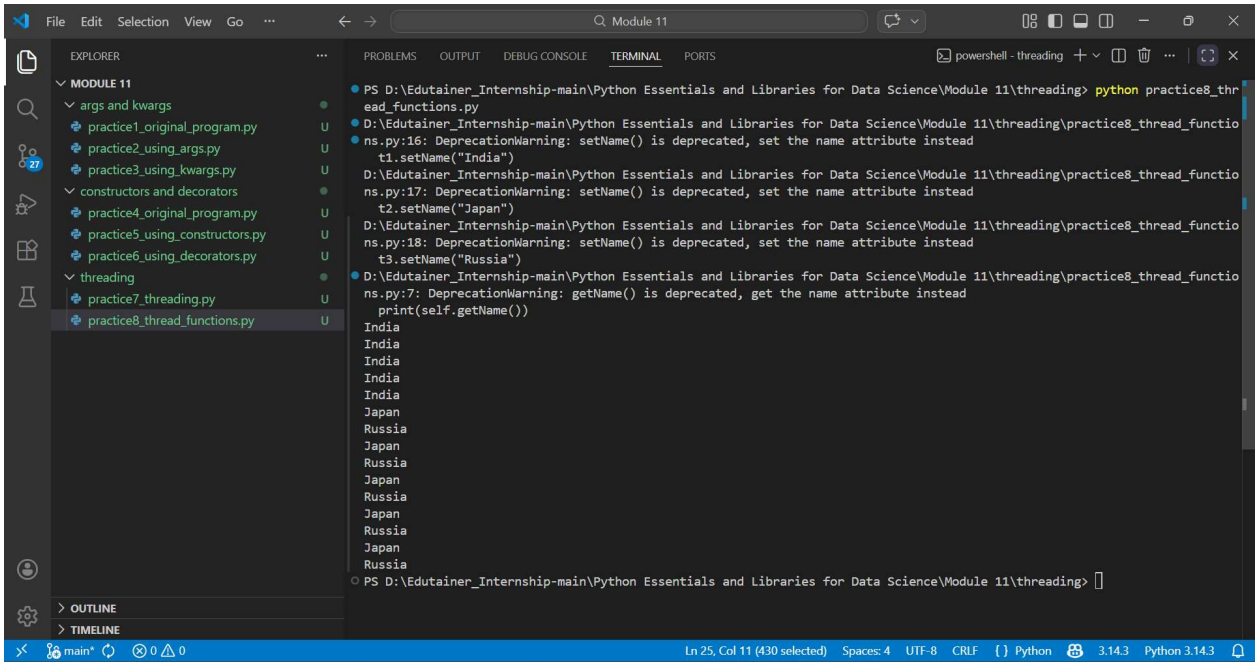
- **import time module**
  → Used to create delay and give animation effect

    - **sleep()**

```
import threading as th
```

```python
import time


class mythread(th.Thread):

    def run(self):

        for i in range(5):

            print(self.getName())

            time.sleep(2)


# create threads

t1 = mythread()

t2 = mythread()

t3 = mythread()


# set thread names

t1.setName("India")

t2.setName("Japan")

t3.setName("Russia")


# start threads

t1.start()

t1.join()   # ensures t1 finishes first


t2.start()
```

```
t3.start()
```


File Edit Selection View Go
Module 11
EXPLORER
MODULE 11
args and kwargs
practice1_original_program.py
practice2_using_args.py
practice3_using_kwargs.py
constructors and decorators
practice4_original_program.py
practice5_using_constructors.py
practice6_using_decorators.py
threading
practice7_threading.py
practice8_thread_functions.py
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell - threading
PS D:\Edutainer_Internship-main\Python Essentials and Libraries for Data Science\Module 11\threading> python practice8_thread_functions.py
D:\Edutainer_Internship-main\Python Essentials and Libraries for Data Science\Module 11\threading\practice8_thread_functions.py:16: DeprecationWarning: setName() is deprecated, set the name attribute instead
t1.setName("India")
D:\Edutainer_Internship-main\Python Essentials and Libraries for Data Science\Module 11\threading\practice8_thread_functions.py:17: DeprecationWarning: setName() is deprecated, set the name attribute instead
t2.setName("Japan")
D:\Edutainer_Internship-main\Python Essentials and Libraries for Data Science\Module 11\threading\practice8_thread_functions.py:18: DeprecationWarning: setName() is deprecated, set the name attribute instead
t3.setName("Russia")
D:\Edutainer_Internship-main\Python Essentials and Libraries for Data Science\Module 11\threading\practice8_thread_functions.py:7: DeprecationWarning: getName() is deprecated, get the name attribute instead
print(self.getName())
India
India
India
India
India
Japan
Russia
Japan
Russia
Japan
Russia
Japan
Russia
Japan
Russia
PS D:\Edutainer_Internship-main\Python Essentials and Libraries for Data Science\Module 11\threading>
main*
Ln 25, Col 11 (430 selected)
Spaces: 4
UTF-8
CRLF
Python
3.14.3
Python 3.14.3